Лабораторная работа «Документирование веб-API»

Лабораторный практикум по ТРПО

1. Описание API с помощью формата описания
2. Проектирование практичного API
3. Контекстное проектирование API

ЧувГУ, каф. Вычислительной техники
Доц. , к.т.н. Ржавин В.В.

Описание API с помощью формата описания
(Разработка спецификации OpenAPI)

Покупая новое устройство с новым для вас GUI, вы либо пробуете его изучить интуитивно, либо обращаетесь к инструкции, где подробно расписано, какой элемент за что отвечает. Также и с API. У хороших API всегда будет документация, в которой описаны все его элементы и способы взаимодействия с ними.

Чтобы API можно было использовать, у него должна быть документация. В документации указано, среди прочего, какие типы запросов будет принимать API, что может делать API, как он форматирует свои ответы и каковы его конечные точки. Разработчики могут просматривать документацию API и использовать эту информацию при создании своих приложений. А начинать надо с разработки спецификации.

Для формирования спецификации можно взять любой редактор, но чаще всего используют редактор Swagger в сервисе SwaggerHub, который доступен для бесплатного использования.

Заготовка спецификации сформируется автоматически в формате YAML и все, что нужно сделать – это исправить ее по своему проекту. Для этого надо внести своих пользователей (actors), свои HTTP-методы и структуры данных.



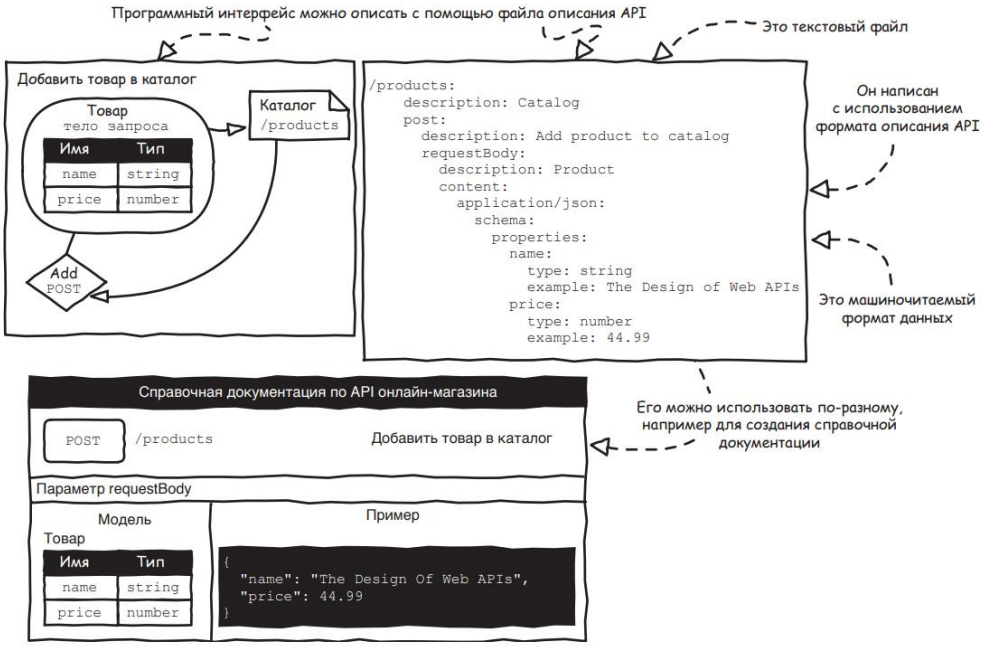
ЧТО НАДО ЗНАТЬ

1. Описание API с помощью формата описания

Если необходимо точное описание программного интерфейса, в особенности его данных, проще и эффективнее использовать структурированный инструмент, такой как формат описания API.

1.1. Что такое формат описания API?

Спецификация OpenAPI (OAS)– это популярный формат описания REST API.



Такой файл можно быстро написать с помощью простого текстового редактора. Но самое главное – поскольку этот файл содержит структурированные данные, программы могут читать его и с легкостью преобразовывать в нечто другое.

Основную причину реализации OpenAPI все хорошо знают. Спецификация определяет стандартное, не зависящее от языка программирования описание интерфейса для HTTP API, которое позволяет людям и компьютерам обнаружить и понять возможности сервиса, не требуя доступа к исходному коду, дополнительной документации или изучения сетевого трафика. При правильном определении через OpenAPI потребитель может понять и взаимодействовать с удалённым сервисом с минимальным количеством логики реализации.

Текущая версия OpenAPI v3.1.0 принята в феврале 2021 года.

Листинг 3.1. Пример описания каталога в формате OpenAPI (OAS)

```
/products:
   description: Catalog
   post:
       description: Add product to catalog
       requestBody:
         description: Product
         content:
           application/json:
             schema:
                properties:
                   name:
                      type: string
                      example: The Design of Web APIs
                   price:
                      type: number
                      example: 44.99
```

Согласно описанию в верхней части документа ресурс /products представляет каталог. Он содержит HTTP-метод POST, который мы можем использовать для добавления товара в каталог. Этой операции нужно `requestBody`, содержащее название и цену.

Описание также предоставляет дополнительную информацию, такую как значения свойств.

Основное использование – создание справочной документации, которая описывает все цели API. Вы можете поделиться им со всеми, кто вовлечен в проект, чтобы они имели представление о создаваемом дизайне.

1.1.1. Спецификация OpenAPI (OAS)

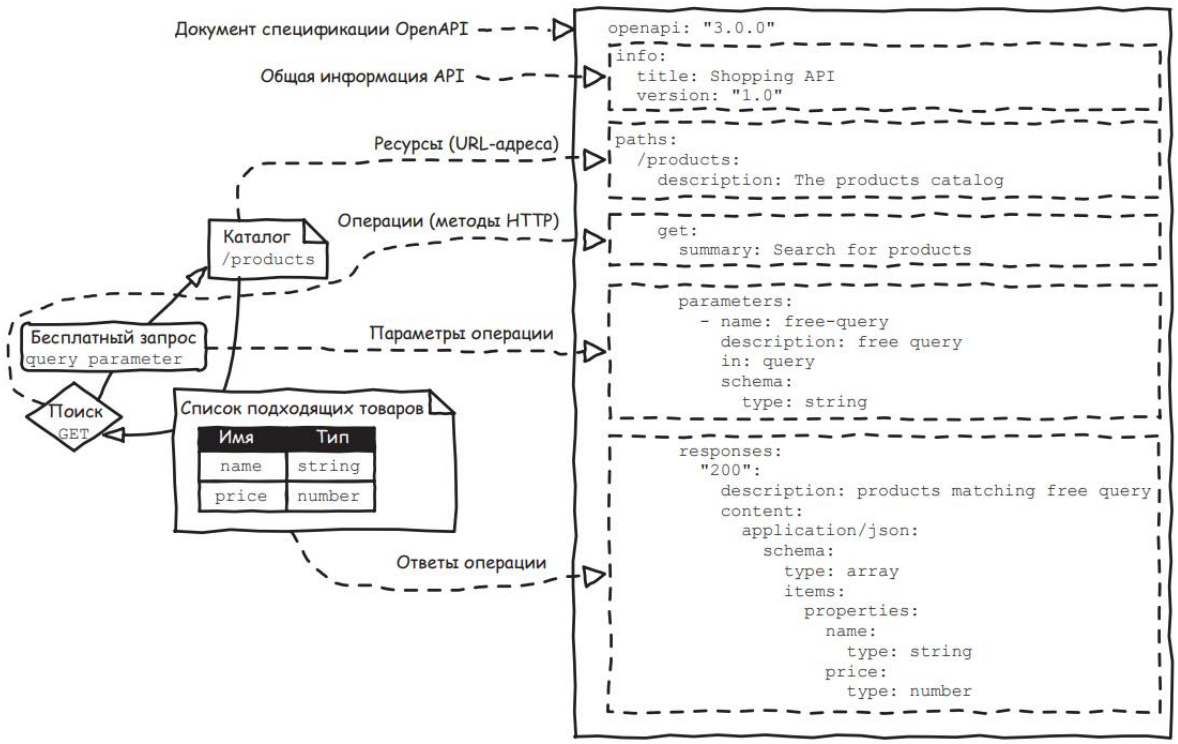
Спецификация OpenAPI (OAS) (Спецификация OpenAPI v3.1.0 | Введение, определения и многое другое (openapis.org)) определяет стандартное, не зависящее от языка программирования описание интерфейса для HTTP API, которое позволяет как людям, так и компьютерам обнаруживать и понимать возможности службы, не требуя доступа к исходному коду, дополнительной документации или проверки сетевого трафика. При правильном определении потребитель может понимать удаленную службу и взаимодействовать с ней с минимальным количеством логики реализации.

OpenAPI рассматривается как универсальный интерфейс для пользователей (клиентов) по взаимодействию с сервисами (серверами). Если спроектирована спецификация для некоторого сервиса, то на её основании можно генерировать исходный код для библиотек клиентских приложений, текстовую документацию для пользователей, варианты тестирования и др. Для этих действий имеется большой набор инструментов для различных языков программирования и платформ.

Спецификация декларативна, и поэтому может быть использована клиентами без знаний особенностей серверной реализации. При этом работать с OpenAPI могут как разработчики, так и рядовые пользователи через готовые инструменты и предоставляемые интерфейсы. В качестве формата используется XML и JSON, но в общем случае может быть выбран и другой язык разметки, например YAML.

Этот файл JSON (часто выражается в YAML вместо JSON) описывает каждую часть API. Описывая API в стандартном формате, инструменты публикации могут программно анализировать информацию об API и отображать каждый компонент в стилизованном интерактивном виде.

На рисунке ниже показан очень простой документ OAS версии 3.0. Этот документ написан с использованием формата данных YAML и описывает цель «Поиск товаров».



Элементы OpenAPI — это paths, parameters, responses и security. Каждый из этих элементов является объектом JSON, который содержит свойства и массивы.

Как и JSON, YAML (http://yaml.org) является форматом данных типа ключ/значение. Описание данного формата приведено в Приложении С. Введение в YAML доступно по адресам YAML Ain't Markup Language (YAML™) версия 1.2.2 и Обзор руководства OpenAPI 3.0 | learnapidoc-ru (starkovden.github.io).

Еще раз отметим, что созданный документ OpenAPI является объектом JSON, но у нас есть возможность выразить JSON с использованием синтаксиса JSON или YAML. Можно рекомендовать использовать формат YAML, который легче читать и писать.

Надо отметить, что кроме OAS существуют и другие форматы описания REST API; наиболее заметными конкурентами OAS являются RAML и Blueprint.

Документ OAS – это простой текстовый файл, который легко можно сохранить в системе управления версиями, такой как Git, как и код. Таким образом, управлять версиями и отслеживать изменения будет просто. Документ OAS имеет структуру, которая помогает более эффективно описывать программный интерфейс. Вы должны описать ресурсы, операции, параметры и ответы. Вы можете определить повторно используемые компоненты (например, модель данных), избегая болезненного и рискованного искусства копирования и вставки фрагментов описаний API.

После того, как получился действующий документ по спецификации OpenAPI, описывающий API, можно “скормить” эту спецификацию различным инструментам, чтобы проанализировать ее и сгенерировать интерактивную документацию. Наиболее

распространенным инструментом, используемым для анализа спецификации OpenAPI, является Swagger UI.

При создании спецификации OpenAPI вместо того, чтобы работать в текстовом редакторе, лучше сразу написать свой код в редакторе Swagger. Редактор Swagger динамически проверяет контент, чтобы определить, является ли созданная спецификация валидной.

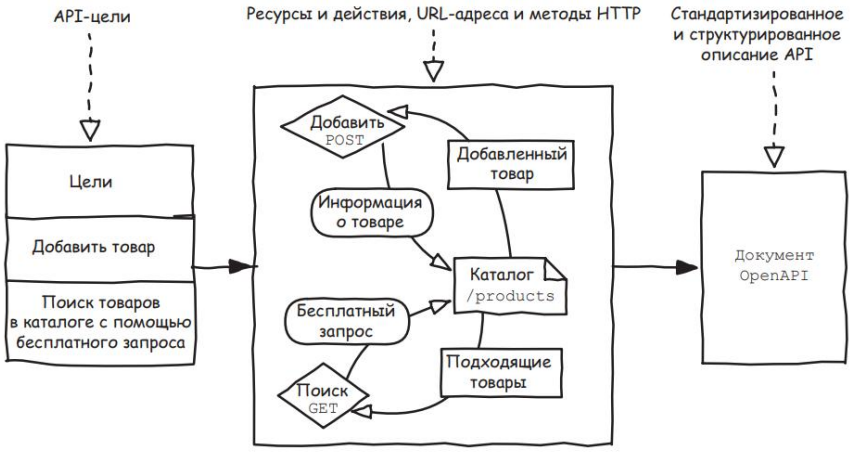
Помимо Swagger UI есть и другие инструменты, которые могут анализировать нашу документацию OpenAPI. Вот список из нескольких инструментов: Restlet Studio, Apiary, Apigee, Lucybot, Gelato, Readme.io, swagger2postman

Обратите внимание, что существуют инструменты проектирования API, которые позволяют описывать программный интерфейс без написания кода. Если вы хотите использовать такой инструмент – отлично; просто убедитесь, что вашу работу можно экспортировать в известный и используемый формат описания API. Но даже если такие инструменты и существуют, все же стоит знать, как написать документ OAS.

Генерация кода

Как только API будет описан с помощью формата описания API, из него можно частично сгенерировать код реализации. Вы получите пустой костяк исходного кода, который также можно использовать для создания рабочего макета. Потребители также могут воспользоваться преимуществами таких машиночитаемых описаний API для генерации кода для использования API. И такой формат также может использоваться инструментами тестирования API или безопасности, а также многими другими инструментами, связанными с API. Например, большинство решений для API-шлюзов (прокси, созданные для предоставления доступа и защите API) можно настроить с помощью файла описания API, такого как документ OAS.

1.2. Описание ресурсов и действий API с помощью OAS



Полученный документ будет содержать точно такую же информацию, что и соответствующий рисунок, но на этот раз информация будет представлена в виде данных в структурированном документ.

1.2.1. Создание документа OAS

В приведенном ниже листинге показан минимальный, но допустимый документ OAS. Этот документ написан с использованием OAS версии 3.0.0. Он описывает API под названием Shopping API в версии 1.0.

Листинг 4.1. Минимальный, но допустимый документ OAS

```
openapi: «3.0.0» ①
info: ②
   title: Shopping API
   version: «1.0»
paths: {} ③
```

① Версия OAS.

② Общая информация API.

③ Пустые пути

Структура документов OAS может меняться от одной версии к другой, поэтому парсеры используют версию `openapi` для соответствующей адаптации парсинга. Обратите внимание, что номера версий спецификации (openapi) и API (info.version) должны быть заключены в кавычки. В листинге свойство paths показано только для того, чтобы создать документ. (Если оно будет отсутствовать, парсер сообщит об ошибке.) Свойство paths содержит ресурсы, доступные для этого API. Сейчас мы можем установить для него значение {} – так описывается пустой объект в YAML. Непустой объект (например, info) не нуждается в фигурных скобках. Далее мы начнем заполнять свойство paths, добавив ресурс.

1.2.2. Описание ресурса

Листинг 4.2. Описание ресурса

```
openapi: «3.0.0»
info:
   title: Shopping
  API version: «1.0»
paths:①
   /products: ②
       description: The products catalog ③
```

① Ресурсы API.

② Путь к ресурсу.

③ Описание ресурса.

1.2.3. Описание операций в ресурсе

Листинг 4.3. Описание действия над ресурсом

```
openapi: «3.0.0»
info:
  title: Shopping API
  version: «1.0»
paths:
  /products: ①
    description: The products catalog
    get: ②
      summary: Search for products ③
      description: | ④
      Search for products in catalog
      using a free query parameter
```

① Ресурс.

② HTTP-метод действия.

③ Краткое описание действия.

④ Длинное описание действия

Листинг 4.4. Описание ответов на действия

```
openapi: «3.0.0»
info:
  title: Shopping API
  version: «1.0»
paths:
  /products:
    description: The products catalog
    get:
      summary: Search for products
      description: | Search for products in catalog using a free query parameter
      responses: ①
        "200": ②
          description: | ③
          Products matching free query parameter
```

① Список ответов.

② Код ответа 200 OK.

③ Описание ответ

Листинг 4.5. Описание еще одного действия

```
openapi: "3.0.0"
info:
   title: Shopping API
   version: «1.0»
paths:
   /products: ①
      description: The products catalog
      get:
         summary: Search for products
         description: |
            Search for products in catalog
            using a free query parameter
         responses:
            "200":
               description: |
               Products matching free query parameter
post: ②
   summary: Add product ③
   description: | ④
         Add product (described in product info
         ➥ parameter) to catalog
responses: ⑤
   "200": ⑥
      description: | ⑦
         Product added to catalog
```

① **Ресурс.**
② **HTTP-метод действия.**
③ **Краткое описание действия.**
④ **Длинное описание действия.**
⑤ **Список ответов.**
⑥ **Ответ 200 OK.**
⑦ **Описание ответа 200 OK.**

1.3. Описание данных API с помощью OpenAPI и JSON Schema

1.3.1. Описание параметров запроса

Листинг 4.6. Описание параметров

```
openapi: "3.0.0"
info:
   title: Shopping API
   version: "1.0"
paths:
   /products: ①
      get: ②
         summary: Search for products
            description: | Search for products in catalog
               ➥ using a free query parameter
               parameters: ③
               [...]
         responses:
            "200":
               description: | Products matching free query
               ➥ parameter
```

① Ресурс.
② Действие.
③ Список параметров действия (за исключением тела).

Листинг 4.7. Описание параметра запроса

```
parameters:
   - name: free-query ①
      description: |  ②
         A product's name, reference, or partial
         ➥ description
         in: query ③
         required: false ④
         schema: ⑤
            type: string ⑥
```

① Имя параметра.
② Описание параметра.
③ Расположение параметра.
④ Является ли параметр обязательным.
⑤ Описание структуры данных параметра.
⑥ Тип параметра (строка).

1.3.2. Описание данных с помощью JSON Schema

Листинг 4.8. Описание базового товара с помощью JSON Schema

```
type: object ①
properties: ②
   reference: ③
      type: string ④
   name: ③
      type: string ④
   price: ③
      type: number ④
```

① Эта схема описывает объект.
② Здесь содержатся свойства.
③ Имя свойства.
④ Тип свойства.

Листинг 4.9. Обязательные и необязательные свойства товара

```
type: object
required: ①
   - reference
   - name
   - price
properties:
   reference: ②
      type: string
   name: ②
      type: string
   price: ②
      type: number
   description: ③
      type: string
```

① Список обязательных свойств.
② Обязательные свойства.
③ Необязательное свойство.

Листинг 4.10. Документирование схемы JSON

```
type: object
description: A product ①
required:
   - reference
   - name
   - price
properties:
   reference:
      type: string
      description: Product's unique identifier ②
      example: ISBN-9781617295102 ③
   name:
      type: string
      example: The Design of Web APIs ③
   price:
      type: number
      example: 44.99 ③
   description:
      type: string
      example: A book about API design ③
```

① **Описание объекта.**
② **Описание свойства.**
③ **Пример значения свойства.**

Листинг 4.11. Описание комплексного свойства с помощью JSON Schema

```
type: object
description: A product
required:
   - reference
   - name
   - price
   - supplier ①
properties:
   reference:
      type: string
      description: Product's unique identifier
      example: ISBN-9781617295102
name:
   type: string
   example: The Design of Web APIs
price:
   type: number
   example: 44.99
description:
   type: string
   example: A book about API design
supplier: ②
   type: object

   description: Product's supplier
   required: ③
      - reference
      - name
   properties: ④
      reference:
         type: string
         description: Supplier's unique identifier
         example: MANPUB
      name:
         type: string
         example: Manning Publications
```

① Необходимо указать поставщика.
② Свойство объекта supplier.
③ Необходимые свойства.
④ Описание свойств.

1.3.3. Описание ответов

Листинг 4.12. Описание данных ответа

```
openapi: "3.0.0"
info:
   title: Shopping API
   version: "1.0"
paths:
   /products:
      get:
      summary: Search for products
      description: | Search using a free query (query parameter)
      parameters:
         [...]
      responses:
         "200":
            description: Products matching free query
            content: ①
               application/json: ②
                  schema: ③
                     [...]
```

① **Определение тела ответа.**
② **Медиа тип тела ответа.**
③ **JSON-схема тела ответа.**

Листинг 4.13. Описание массива товаров

```
responses:
   "200":
      description: Products matching free query
      content:
         application/json: ①
            schema: ②
               type: array ③
               description: Array of products
               items: ④
                  type: object
                  description: A product
                  required:
                     - reference
                     - name
                     - price
                     - supplier
               properties:
                  reference:
                     description: Unique ID identifying a product
                     type: string
                  name:
                     type: string
                  price:
                     description: Price in USD
                     type: number
                  description:
                     type: string
                  supplier:
                     type: object
                     description: Product's supplier
                     required:
                        - reference
                        - name
                     properties:
                        reference:
                           type: string
                        name:
                           type: string
```

① Медиа тип тела ответа.
② JSON-схема тела ответа.
③ Тип ответа – массив.
④ Схема элементов массива.

Листинг 4.14. Пример массива товаров в формате JSON

```
[
   {
      "reference": "123-456",
      "name": "a product",
      "price": 9.99,
      "supplier": {
         "reference": "S789",
         "name": "a supplier"
      }
   },
   {
      "reference": "234-567",
      "name": "another product",
      "price": 19.99,
      "supplier": {
         "reference": "S456",
         "name": "another supplier"
      }
   }
]
```

1.3.4. Описание параметров тела

Листинг 4.15. Описание параметра тела действия

```
openapi: "3.0.0"
   info:
   title: Shopping API
version: "1.0"
paths:
   /products:
   description: The products catalog
   [...]
   post:
      summary: Add product
      description: Add product to catalog
      requestBody: ①
         description: Product's information ②
         application/json: ③
            schema: ④
               [...]
      responses:
         "200":
            description: Product added to catalog
```

① Определение параметра тела.
② Описание параметра тела.
③ Медиа тип параметра тела.
④ Схема параметра тела.



ЗАДАНИЕ

1. Сформируйте JSON-структуры данных (см. Приложение А)

Метод	Конечная точка (endpoint)	Параметры ()	Ответы (response)
GET			
			GET с параметрами фильтрации
…	…	…	…

2. В редакторе https://editor-next.swagger.io/ составьте вручную спецификацию OpenAPI для одной конечной точки. Используя Учебник OpenAPI шаг за шагом, пройдите через все 8 этапов создания документа в спецификации OpenAPI.

   Фреймворк Swagger от компании SmartBear позволяет формировать спецификацию OpenAPI 3 и интерактивно ее просматривать в визуальном веб-GUI. Спецификация OpenAPI описывает поддерживаемые REST-приложением HTTP-методы и структуры используемых данных. Редактор Swagger поддерживает работу с форматами JSON И YAML, проверяет валидность созданной спецификации и предоставляет GUI для тестирования HTTP-методов, объявленных в спецификации.

   Напоминаю ещё раз, что сам документ OAS представляет собой JSON объект, который может быть представлен в формате текстового файла JSON или YAML, последнее является более строгим и компактным синтаксисом JSON объекта. Сам же сайт Swagger предоставляет веб-интерфейс для написания и редактирования OAS в формате YAML с последующей возможностью экспорта спецификации в JSON формат.

   Источники

   - Учебник OpenAPI шаг за шагом
   - Онлайн-редактор https://editor.swagger.io/ или https://editor-next.swagger.io/

3. На основе разработанной выше спецификации OpenAPI создайте свою веб-страницу с интерактивной документацией с помощью фреймворка Swagger UI.

   Пользовательский интерфейс Swagger позволяет любому человеку — будь то команда разработчиков или конечные потребители — визуализировать ресурсы API и взаимодействовать с ними без какой-либо логики реализации. Он автоматически генерируется на основе спецификации OpenAPI (ранее известного как Swagger), а визуальная документация упрощает его внедрение на серверной стороне и использование на стороне клиента.

   Источники

   - Руководство Swagger UI - Руководство Swagger UI | learnapidoc-ru (starkovden.github.io)

4. **Тестирование и отладка API**. Swagger позволяет системному аналитику тестировать и отлаживать API, используя интерактивную консоль Swagger UI. Аналитик может отправлять запросы к API, проверять ответы и убедиться, что API работает корректно и соответствует требованиям.

5. **Генерация кода**. Также стоит упомянуть, что Swagger позволяет сгенерировать непосредственно код клиента или сервера по имеющейся OAS, для этого нужен генератор кода Swagger-Codegen. Генерируемый код получается весьма неказистым и проще написать его самостоятельно.

ТРЕБОВАНИЯ К ОТЧЕТУ

- JSON-структуры данных.
- Спецификация OpenAPI для одной конечной точки.
- Веб-страница с интерактивной документацией (с помощью фреймворка Swagger UI).



- Что означает «API-first подход»?
- Назовите типы веб-API
- Что означает слово REST?
- Что такое веб-сервис? Какой тип API использует веб-сервис?
- Какие функции выполняет API-шлюз?
- Почему REST не является протоколом?
- Почему архитектура, основанная на событиях, не будет удовлетворять требованиям RESTful?
- Назовите основные принципы REST.
- Какую цель преследует требование «RESTful-система должна производить операции в клиент-серверной модели»?
- В чем смысл требования «Отсутствие состояния» в RESTful-системе?
- Принцип унификация или единообразие интерфейса. Как достигается унификация?
- Назовите четыре составляющих унифицированного интерфейса.
- Как реализуется унификация интерфейса в REST?
- Почему возникла необходимость разработки ключа API? В чем недостатки базовой аутентификации?
- Есть ли альтернативы REST?

ИСТОЧНИКИ

Рекомендуемая литература

- Арно Лоре «Проектирование веб-API»

Сайты

- Курс по документированию REST API | learnapidoc-ru (starkovden.github.io)
- Руководство OpenAPI Шаг 1 Объект "openapi" | learnapidoc-ru (starkovden.github.io)
- Подходы к проектированию RESTful API / Хабр (habr.com)
- Знакомство со спецификациями OpenAPI и Swagger | learnapidoc-ru (starkovden.github.io)
- Учебник OpenAPI шаг за шагом learnapidoc-ru (starkovden.github.io)
- Проектирование CRUD API и рекомендации по CRUD API (stoplight.io)
- Как читать документацию по API | Авто стоп-сигнал (stoplight.io)
- Оцените свою документацию по API с помощью этого руководства | (stoplight.io)

- Использование схемы JSON для пользовательских ответов API | (stoplight.io)
- Редактор JSON - редактируйте код JSON за секунды (jsononline.net)
- Как создать свой Swagger на REST-приложение в Google Colab (babok-school.ru)
- Проектируем и документируем REST: спецификация OpenAPI (babok-school.ru)
- YAML Ain't Markup Language (YAML™) версия 1.2.2
- Как создать свою спецификацию OpenAPI | ЕСКС (gosuslugi.ru)
- Полное руководство по разработке API | AppMaster
- Рекомендации по проектированию веб-API - Azure Architecture Center | Microsoft Learn
- OpenAPI 3.1.0 (русская редакция) - maksyutin.ru / Разработка ПО: взгляд изнутри
- Swagger (OpenAPI 3.0) / Хабр (habr.com)
- Визуальное проектирование API для OpenAPI | Поддержка конструктора схем API JSON | Авто стоп-сигнал (stoplight.io) 14-дневная бесплатная пробная версия.
- OpenAPI Generator - Генерация кода по спецификации (yandex.ru)
- Учебное пособие по генератору OpenAPI (практическое) (apidog.com)
- Шаг 1 "Описание ресурса (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Шаг 2 "Конечные точки и методы (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Шаг 3 "Параметры (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Шаг 4 "Пример запроса (Описание API)" | learnapidoc-ru (starkovden.github.io)
- 5 "Пример и схема ответа (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Собираем все вместе (Описание API) | learnapidoc-ru (starkovden.github.io)
- О модуле "Спецификация OpenAPI и Swagger" | learnapidoc-ru (starkovden.github.io)

Видео

- Знакомство со Swagger документацией. Разбор основных методов запросов
- Как использовать swagger
- Первый опыт в OpenAPI / Swagger - это PetStore проект
- OpenAPI и Swagger Editor - своё описание REST API с нуля
- OpenAPI Generator - Генерация кода по спецификации
- (11) Составление спецификаций. Теория. Surf BA School - YouTube
- (11) Составление спецификаций. Практика. Surf BA School - YouTube
- https://yandex.ru/video/preview/7438056860438307276

Инструменты

- Введение | AppMaster
- Решения для проектирования и управления API (stoplight.io)
- Авто стоп-сигнал (stoplight.io)
- Apidog An integrated platform for API design, debugging, development, mock, and testing

ПРИЛОЖЕНИЕ

A. YAML (YAML — Википедия (wikipedia.org))

YAML (произносится как "ямл") – удобный для восприятия формат сериализации данных. Как и JSON, YAML (http://yaml.org) является форматом данных типа ключ/значение. YAML ориентирован на удобство ввода-вывода типичных структур данных многих языков программирования. На рисунке показано сравнение двух этих форматов.



YAML в сравнении с JSON

Обратите внимание на следующие моменты:

- имена и значения свойств в YAML не заключены в двойные кавычки ("");
- структурные фигурные скобки ({}) и запятые (,) в JSON заменены символами перевода строки и отступами в YAML;
- скобки массива ([]) и запятые (,) заменены черточками (-) и символами перевода строки;
- в отличие от JSON, YAML допускает комментарии, начинающиеся с символа #. Преобразовать один из этих форматов в другой можно относительно легко. Однако имейте в виду, что при преобразовании документа из формата YAML в JSON комментарии будут утеряны;
- каждый уровень в YAML (определенный отступом в два пробела) является объектом.

Существует множество видов структур данных, но все они могут быть адекватно представлены тремя основными примитивами:

1. **отображениями (хеши/словари),**
2. **последовательностями (массивы/списки) и**
3. **скалярами (строки/числа).**

YAML использует эти примитивы. Однако, поскольку спецификация OpenAPI является объектом JSON, YAML использует терминологию JSON - например, «объекты», «массивы», «свойства», «поля» и так далее. YAML добавляет простую систему типизации и механизм псевдонимов для формирования полноценного языка для сериализации любых машинных данных структура.

В то время как большинство языков программирования могут использовать YAML для сериализации данных, YAML преуспевает в работе с теми языками, которые фундаментально

построены вокруг трех основных примитива. К ним относятся распространенные динамические языки, такие как JavaScript, Perl, PHP, Python и Ruby.

В трактовке названия отражена история развития: на ранних этапах YAML расшифровывался как Yet Another Markup Language («Ещё один язык разметки») и даже позиционировался как конкурент XML, но позже был переименован (***YAML Ain't Markup Language*** - это не язык разметки) с целью акцентировать внимание на данных, а не на разметке документов.

B. Общая структура yaml документации

```
openapi: 3.0.1 (1)
info: <6 keys> (2)
servers: <1 item> (3)
tags: (4)
  - name: 1.    Article → Habr. Получение всех статей
    description: |     ---…*     :::
paths: (5)
  /articles/:
    get: <6 keys>
    post: <8 keys>

  /articles/{articleId}:
    get: <6 keys>
    put: <7 keys>
    delete: <6 keys>

components: (6)
  schemas:
    ArticleDto:
      type: object
      properties: <4 keys>
    Hubs:
      type: string
      enum: <3 items>
  securitySchemes: <1 key>
security: (7)
  - Bearer: []
```

1. `openApi` - Содержит версию
2. `info` - метаданные для api
3. `servers` - информация о серверах с api
4. `tags` - дополнительные метаданные, в этом блоке нужно писать текст ФТ и регламента взаимодействия
5. `paths` - описание endpoints и интерфейсов контроллеров
6. `components` - описание модели данных

7. `security` - схема безопасности

Исходный код на **github** и пример документации можно посмотреть **здесь**

